

DATA NORMALIZATION /SCALING

DATA NORMALIZATION/SCALING

What is Normalization?

- Normalization is a specific form of feature **scaling** that transforms the range of features to a standard scale.
- Normalization is required only when your dataset has features of varying ranges
- Data normalization is the process of reorganizing data within a database so that users can utilize it for further queries and analysis.
- Simply put, it is the process of developing clean data. This includes eliminating redundant and unstructured data and making the data appear similar across all records and fields.

Why Normalize Data?

- Normalized data enhances model performance and improves the accuracy of a model. It aids algorithms that rely on **distance** metrics, such as [k-nearest neighbors](#) by preventing features with larger scales from dominating the learning process.
- Normalization fosters stability in the optimization process, promoting faster convergence during gradient-based training. It mitigates issues related to vanishing or exploding gradients, allowing models to reach optimal solutions more efficiently.
- Normalized data is also easy to interpret and thus, easier to understand. When all the features of a dataset are on the same scale, it also becomes easier to identify and visualize the relationships between different features and make meaningful comparisons.

EXAMPLE

- Square footage: Ranges from 500 to 5000 square feet
- Number of bedrooms: Ranges from 1 to 5
- Distance to supermarket: Ranges from 0.1 to 10 miles

EXAMPLE (CONT)

- **Algorithm might give more weight to features with large scales**, such as square footage. During training, the algorithm assumes that a change in total square footage will have a significant impact on housing prices.
- The algorithm might **overlook** the nuances of features that are relatively small, such as the number of bedrooms and distance to the supermarket. This skewed emphasis can lead to suboptimal model performance and biased predictions.
- By normalizing the features, we can ensure that **each feature contributes proportionally** to the model's learning process. The model can now learn patterns across all features more effectively, leading to a more accurate representation of the underlying relationships in the data.

- **Ensure equal feature importance:**

- Normalization scales features so that one feature with a large range (e.g., income) doesn't unfairly dominate other features with smaller ranges (e.g., age) in distance-based calculations.
- If one feature ranges from 0–1 and another from 0–10,000, many models will treat the larger-scale feature as more important, even if it isn't. Normalization prevents this.

- **Enhance accuracy:**

- For certain models, normalization can lead to more accurate predictions by preventing features with large values from disproportionately influencing the model.

- **Improve algorithm performance:**

- To make distance-based algorithms work correctly: Algorithms like k-NN, k-means, and SVM rely on distance. Without normalization, distances become skewed and the model performs poorly.

- **Speed up training**

Normalization Techniques

TS. Nguyen Quoc Minh

MIN-MAX NORMALIZATION (RESCALING)

Normalized Value

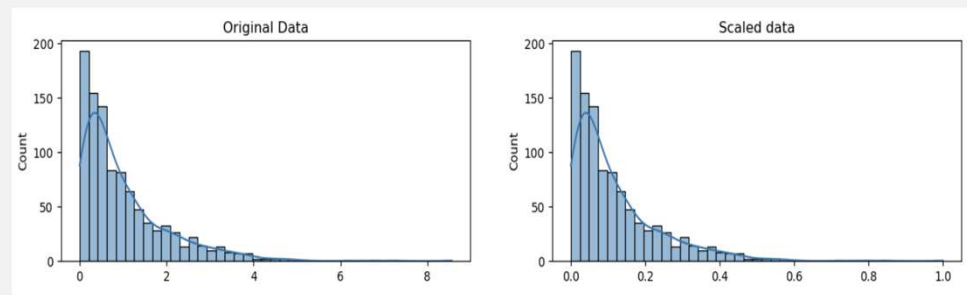
Original Value

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Maximum Value of x

Minimum Value of x

Formula for Min-Max Normalization



- Min-max scaling is very often simply called 'normalization.'
- It transforms features to a specified range, typically [0 - 1]

Min-max scaling is a good choice when:

- The approximate upper and lower bounds of the dataset is known, and the dataset has few or no outliers
- When the data distribution is unknown or non-Gaussian, and the data is approximately uniformly distributed across the range
- When maintaining the distribution's original shape is essential

Z-SCORE NORMALIZATION (STANDARDIZATION)

- Z-score normalization (standardization) assumes a Gaussian (bell curve) distribution of the data and transforms features to have a mean (μ) of 0 and a standard deviation (σ) of 1.

The diagram shows the formula for standardization: $x' = \frac{x - \mu}{\sigma}$. Arrows point from labels to the corresponding parts of the formula: 'Standardised Value' points to x' , 'Original Value' points to x , 'Sample Mean' points to μ , and 'Sample Standard Deviation' points to σ . Below the formula, the text 'Formula for Standardisation' is written.

$$x' = \frac{x - \mu}{\sigma}$$

Formula for Standardisation

- This technique is particularly useful when dealing with algorithms that assume normally distributed data, such as many linear models. (e.g., linear models, logistic regression)

MAXABS SCALING

$$x' = \frac{x}{|x|_{\max}}$$

- Scales to **[-1, 1]** but keeps sparsity (good for sparse matrices)
- Working with sparse data (e.g., text features)

L2 OR L1 NORMALIZATION (VECTOR NORMALIZATION)

$$x' = \frac{x}{\|x\|_2}$$

- Scales each sample (row) so its vector length = 1
- Common in text processing (TF-IDF)
- Need normalized vectors for similarity/distance

Normalization	Standardization
This technique uses minimum and max values for scaling of model.	This technique uses mean and standard deviation for scaling of model.
It is helpful when features are of different scales.	It is helpful when the mean of a variable is set to 0 and the standard deviation is set to 1.
Scales values ranges between [0, 1] or [-1, 1].	Scale values are not restricted to a specific range.
It got affected by outliers.	It is comparatively less affected by outliers.
Scikit-Learn provides a transformer called MinMaxScaler for Normalization.	Scikit-Learn provides a transformer called StandardScaler for Normalization.
It is also called Scaling normalization.	It is known as Z-score normalization.
It is useful when feature distribution is unknown.	It is useful when feature distribution is normal.

ONE HOT ENCODING

- A technique for representing **categorical data** as numerical vectors, where each unique category is represented by a binary column with a value of **1** indicating its presence and **0** indicating its absence.
- A method of converting categorical variables into a format that can be provided to machine learning algorithms to improve prediction.
- It involves creating new binary columns for each unique category in a feature. Each column represents one unique category, and a value of 1 or 0 indicates the presence or absence of that category.

- The original "Color" column is replaced by three new binary columns, each representing one of the colors.
- A value of 1 indicates the presence of the color in that row, while a 0 indicates its absence.

Color		Color	Red	Green	Blue
Red		Red	1	0	0
Green	→	Green	0	1	0
Blue		Blue	0	0	1

Label Encoding

Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50



One Hot Encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

ONE HOT ENCODER

Why Use One-Hot Encoding?

It transforms categorical data into a format that machine learning models can **easily understand** and use. It allows **each category to be treated independently** without implying any false relationships between them.

Machine learning compatibility

Categorical data needs to be transformed into a numerical format for Machine learning algorithms to use effectively

Avoiding ordinality

Label encoding might suggest an order or ranking among categories that doesn't actually exist. For example, assigning 1 to Red, 2 to Green, and 3 to Blue could make the model think that Green is greater than Red and Blue is greater than both.

One-hot encoding solves this problem by creating a separate binary column for each category. So that the model can see that each category is distinct and unrelated to the others.

ADVANTAGES

- It allows the use of categorical variables in models that require **numerical** input.
- It can improve model performance by providing **more information** to the model about the categorical variable.
- It can help to **avoid the problem of ordinality**, which can occur when a categorical variable has a natural ordering (e.g. “small”, “medium”, “large”).

DISADVANTAGES

- It can lead to **increased dimensionality**, as a separate column is created for each category in the variable. This can make the model more complex and slow to train.
- It can lead to **sparse data**, as most observations will have a value of 0 in most of the one-hot encoded columns.
- It can lead to **overfitting**, especially if there are many categories in the variable and the sample size is relatively small.